

SmartBasket

Full-Stack Role-Based E-Commerce Platform

Technical Documentation

April 2026

Prepared By

[NAMRA GAJERA]

Technology Stack

ASP.NET Core 10 | React 18 + TypeScript | MySQL 8 | Entity Framework Core | JWT | AI/ML
Integration

Table of Contents

- 1. Abstract 3**
- 2. Introduction 3**
 - 2.1 Background..... 3
 - 2.2 Purpose of the Project 3
- 3. Problem Statement 4**
- 4. Objectives..... 4**
- 5. System Architecture 5**
 - 5.1 High-Level Architecture 5
 - 5.2 Backend Architecture 5
 - 5.3 Frontend Architecture 5
- 6. Technology Stack 6**
 - 6.1 Backend..... 6
 - 6.2 Frontend 6
 - 6.3 Database 7
 - 6.4 External Services & Tools 7
- 7. Features & Modules 8**
 - 7.1 Customer Module 8
 - 7.2 Admin Module..... 8
 - 7.3 Delivery Module 8
- 8. Database Design 9**
 - 8.1 Entity Summary 9
 - 8.2 Entity Relationships 9
 - 8.3 Order Status Enumeration..... 10
- 9. API Design 11**
 - 9.1 Authentication Endpoints — /api/auth 11
 - 9.2 Product Endpoints — /api/products..... 11
 - 9.3 Order Endpoints — /api/orders..... 11
 - 9.4 Delivery Endpoints — /api/delivery..... 12
 - 9.5 AI Endpoints — /api/ai 12
- 10. Workflow & System Flow..... 13**
 - 10.1 Order Lifecycle Flow..... 13
 - 10.2 Rejection Workflow 13
 - 10.3 Delivery OTP Verification Flow..... 14
 - 10.4 Authentication Flow 14
- 11. Security Implementation..... 15**
 - 11.1 Authentication 15
 - 11.2 Authorization..... 15
 - 11.3 Password Security 15
 - 11.4 Data Protection & Configuration 15
 - 11.5 Runtime Protections 16

12. AI Features	16
12.1 AI Smart Search	16
12.2 AI Product Recommendations.....	17
12.3 AI Description Generator.....	17
12.4 AI Review Summarizer	17
13. Implementation Details	18
13.1 Repository Pattern	18
13.2 DTO Contract Design	18
13.3 Startup Seeding	18
13.4 React Query for State Management.....	18
13.5 OTP Generation Strategy	18
13.6 Stock Restoration on Rejection	18
14. User Interface Overview	19
14.1 Public Storefront	19
14.2 Customer Panel	19
14.3 Admin Panel	19
14.4 Delivery Panel	19
15. Testing	20
15.1 API Testing	20
15.2 Role-Based Access Testing	20
15.3 Frontend Functional Testing.....	20
15.4 Known Test Gaps	20
16. Limitations.....	20
17. Future Scope	21
18. Conclusion	22

List of Tables

Table 1 — Backend Architecture Layers 5

Table 2 — Technology Stack: Backend..... 6

Table 3 — Technology Stack: Frontend 6

Table 4 — Technology Stack: Database 7

Table 5 — External Services and Tools 7

Table 6 — Customer Module Features 8

Table 7 — Admin Module Features..... 8

Table 8 — Delivery Module Features 8

Table 9 — Database Entity Summary 9

Table 10 — Order Status Enumeration 10

Table 11 — Authentication Endpoints (/api/auth) 11

Table 12 — Product Endpoints (/api/products) 11

Table 13 — Order Endpoints (/api/orders)..... 11

Table 14 — Delivery Endpoints (/api/delivery)..... 12

Table 15 — AI Endpoints (/api/ai)..... 12

Table 16 — Runtime Security Protections..... 16

Table 17 — Project Limitations Summary 20

List of Figures

Figure 1 — High-Level System Architecture Diagram 5

Figure 2 — Frontend Source Directory Structure 5

Figure 3 — Database Entity Relationship Diagram 10

Figure 4 — Order Lifecycle State Flow..... 13

Figure 5 — Rejection Workflow State Flow 13

1. Abstract

SmartBasket is a comprehensive, production-grade, role-based e-commerce platform engineered to simulate the operational depth of industry-level commerce systems such as those deployed by Amazon, Flipkart, and similar enterprises. The platform is built on a modern full-stack architecture comprising an ASP.NET Core 10 Web API backend, a React 18 with TypeScript frontend, and a MySQL 8 relational database.

The system supports three distinct operational roles — Customer, Administrator, and Delivery Agent — each with a purpose-built interface and strictly controlled access. Key capabilities include end-to-end order lifecycle management, OTP-based delivery verification, multi-level rejection workflows with complete audit trails, AI-assisted product discovery, and AI-generated content automation.

This document presents a complete technical record of SmartBasket, covering its system architecture, data model, API surface, security implementation, AI feature design, and deployment considerations. It is intended for academic evaluation, portfolio presentation, and internship submission purposes.

2. Introduction

2.1 Background

The e-commerce sector has grown exponentially over the last decade, demanding robust, scalable, and intelligent software platforms. Modern commerce systems are no longer simple product listings; they involve complex workflows spanning inventory management, multi-role access control, logistics coordination, real-time delivery tracking, and AI-powered personalization.

SmartBasket was conceived as a capstone full-stack project that mirrors the structural and operational complexity of a real-world commerce platform. Rather than building a minimal demo, the project intentionally pursues production-level patterns: clean layered architecture, DTO-based API contracts, JWT authentication with refresh tokens, role-gated endpoint access, and AI service integration.

2.2 Purpose of the Project

The primary purpose of SmartBasket is to demonstrate mastery of full-stack software engineering through a working, well-structured application. Specific goals include:

- Designing and implementing a relational data model for a realistic commerce domain
- Building a secure, layered RESTful API with role-based authorization
- Developing a reactive single-page application with modular, feature-first architecture
- Integrating AI/ML capabilities for enhanced product discovery and content automation
- Implementing real-world workflows such as OTP delivery verification and rejection audit trails

3. Problem Statement

Small to mid-scale e-commerce operations commonly face the following challenges:

1. Absence of a unified platform that manages the full order lifecycle — from cart to delivery — across multiple stakeholder roles without redundant or conflicting access controls.
2. Lack of transparent rejection workflows, making it difficult to audit why and when an order was cancelled and by whom.
3. Inefficient delivery operations due to manual assignment, missing OTP-based handover verification, and no structured delivery history.
4. Primitive product discovery that relies purely on keyword matching, failing to surface semantically relevant results or personalized recommendations.
5. Manual and inconsistent product content creation, leading to incomplete or poorly written product descriptions.

SmartBasket addresses each of these challenges through deliberate feature design and architectural decisions detailed in this document.

4. Objectives

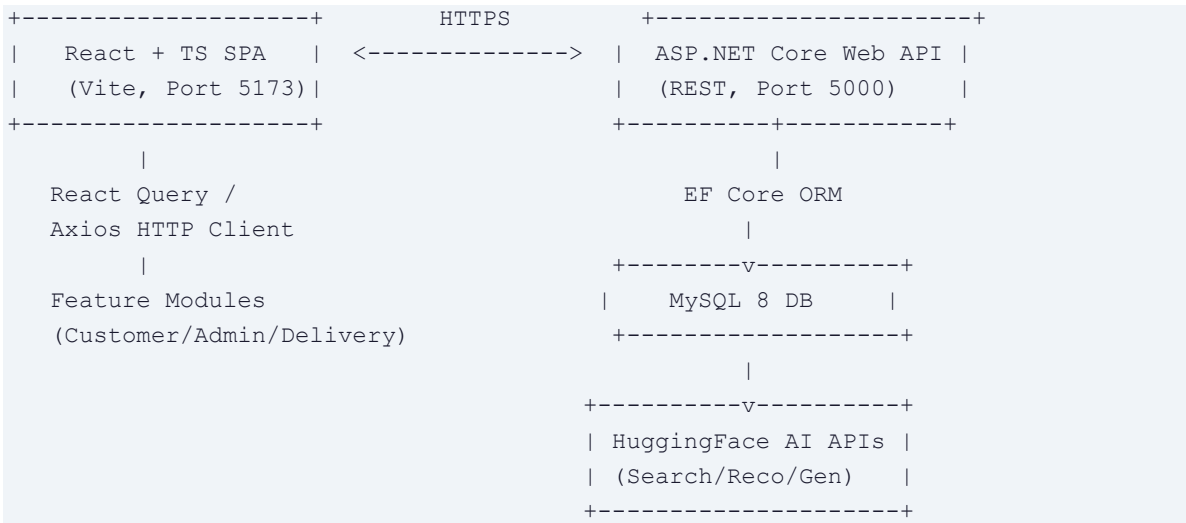
The following objectives guided the design and implementation of SmartBasket:

- Implement a three-role access model (Customer, Admin, Delivery) with JWT-based authentication and attribute-level route protection.
- Deliver complete order lifecycle management covering creation, assignment, delivery, and rejection states.
- Provide OTP-based delivery confirmation to ensure secure handover verification between delivery agent and customer.
- Build a multi-party rejection system with structured metadata capturing the rejecting actor, reason, and timestamp.
- Integrate AI-powered semantic search to improve product discoverability beyond simple keyword matching.
- Provide AI-generated product descriptions and AI-summarized review aggregations to reduce manual content effort.
- Design a clean, reactive SPA frontend with per-role dashboards, consistent UX patterns, and real-time state management via React Query.
- Maintain a separation-of-concerns architecture across Controllers, Services, Repositories, and DTOs in the backend.
- Implement IP-based rate limiting, global exception handling, and response wrapping for production-grade API behavior.

5. System Architecture

5.1 High-Level Architecture

SmartBasket follows a three-tier client-server architecture:



The frontend communicates exclusively with the backend API over HTTP. The backend manages business logic, data access, and AI integrations. The database layer is abstracted through Entity Framework Core using the repository pattern.

Figure 1: High-Level System Architecture Diagram

5.2 Backend Architecture

The backend follows a strictly layered, separation-of-concerns architecture:

Table 1: Backend Architecture Layers

Layer	Responsibility	Location
Controllers	HTTP routing, request validation, role guards, response delegation	Backend/Controllers/
Services	Business logic, orchestration, validation rules	Backend/Services/
Repositories	Data access abstraction over EF Core DbContext	Backend/Repositories/
DTOs	API contract objects — request/response shapes	Backend/DTOs/
Models	EF Core entity classes mapped to database tables	Backend/Models/
Middleware	Global exception handling, API response	Backend/Middleware/

Layer	Responsibility	Location
	wrapping	
AI Layer	HuggingFace API integration for search, reco, generation	Backend/AI/
Data	EF Core DbContext, startup seed and repair tasks	Backend/Data/

5.3 Frontend Architecture

The frontend follows a feature-first modular structure, with shared infrastructure for routing, authentication, and API communication:

```
src/
  api/          -- Axios API client modules, one file per domain
  app/          -- App-level setup and providers
  auth/         -- AuthContext: JWT token management & user session
  components/   -- Shared UI components (Navbar, Sidebar, etc.)
  features/     -- Feature modules:
    home/       -- Landing page, hero, featured products
    products/   -- Product listing, detail, search
    cart/       -- Cart management
    orders/     -- Customer order tracking
    admin/      -- Admin dashboard, categories, products, orders, AI tools
    delivery/   -- Delivery agent order queue and past deliveries
    ai/         -- Smart search UI feature
    auth/       -- Login and registration screens
  layouts/     -- MainLayout, PanelSidebar
  routes/      -- Centralized router configuration
  types/       -- TypeScript type definitions
  utils/       -- Shared utility functions
```

Figure 2: Frontend Source Directory Structure

6. Technology Stack

6.1 Backend

Table 2: Technology Stack: Backend

Technology	Version	Role
.NET / ASP.NET Core	10 (net10.0)	Web API framework
Entity Framework Core	8.0.13	ORM and migrations
Pomelo MySQL Provider	8.0.3	MySQL database driver for EF Core
JWT Bearer Auth	Built-in	Stateless authentication
BCrypt.Net	Latest	Password hashing
Serilog	Latest	Structured logging with file output
Swashbuckle	Latest	Swagger / OpenAPI documentation
Built-in Rate Limiting	.NET 10	IP-based request throttling

6.2 Frontend

Table 3: Technology Stack: Frontend

Technology	Version	Role
React	18	UI component framework
TypeScript	5.9	Static typing across frontend
Vite	5	Build tool and dev server
Tailwind CSS	3	Utility-first CSS framework
React Router	6	Client-side routing
TanStack React Query	5	Server state management and caching
Axios	Latest	HTTP client for API calls
Framer Motion	Latest	UI animation library
Lucide React	Latest	Icon library

6.3 Database

Table 4: Technology Stack: Database

Technology	Version	Role
MySQL	8+	Primary relational database
EF Core Migrations	8.0.13	Schema versioning and migration management
Startup Seeder	Custom	Initial data seeding on application boot

6.4 External Services & Tools

Table 5: External Services and Tools

Tool / Service	Purpose
HuggingFace API	AI model hosting for semantic search, recommendations, and text generation
Swagger UI	Interactive API documentation and testing interface
dotenv	Environment variable management via .env file
Git + .gitignore	Version control with secret file exclusions

7. Features & Modules

7.1 Customer Module

The Customer module delivers the complete shopping experience for end users.

Table 6: Customer Module Features

Feature	Description
Product Browsing	View all products by category with filtering capabilities
Product Detail	Full product page with images, pricing, description, and reviews
AI Smart Search	Semantic search powered by HuggingFace embeddings for intent-based discovery
Cart Management	Add, update quantity, and remove items from a persistent cart
Checkout	Submit order with shipping address from cart state
Order Tracking	View all personal orders and detailed status for each
OTP Visibility	View delivery OTP for orders in Assigned or Out-For-Delivery status
Order Rejection	Cancel an order before delivery completion with a reason
Review System	Create, edit, and delete product reviews post-purchase
Rejection History	View rejection metadata on affected orders

7.2 Admin Module

The Admin module provides complete operational control over the platform.

Table 7: Admin Module Features

Feature	Description
Dashboard Summary	High-level KPIs and platform activity overview
Category Management	Create and browse product categories
Product Management	Full CRUD for products including image upload
Order Management	View all platform orders with full lifecycle visibility
Delivery Assignment	Assign registered delivery agents to pending orders
Admin Rejection	Cancel orders with reason on behalf of the platform
Rejected Orders View	Tabular history of all rejected orders with rejection metadata
Past Deliveries	Historical view of all completed deliveries
AI Description Generator	Generate product descriptions using AI from product attributes

Feature	Description
AI Review Summarizer	Aggregate and summarize customer reviews for a product using AI

7.3 Delivery Module

The Delivery module gives delivery agents a focused workflow interface.

Table 8: Delivery Module Features

Feature	Description
Assigned Orders Queue	View all orders currently assigned to the agent
Status Transitions	Mark orders as Out-For-Delivery or attempt delivery completion
OTP Verification	Enter customer OTP to confirm and complete a delivery
Delivery Rejection	Reject an assigned order with a documented reason
Past Deliveries	Historical view of all completed deliveries for the agent

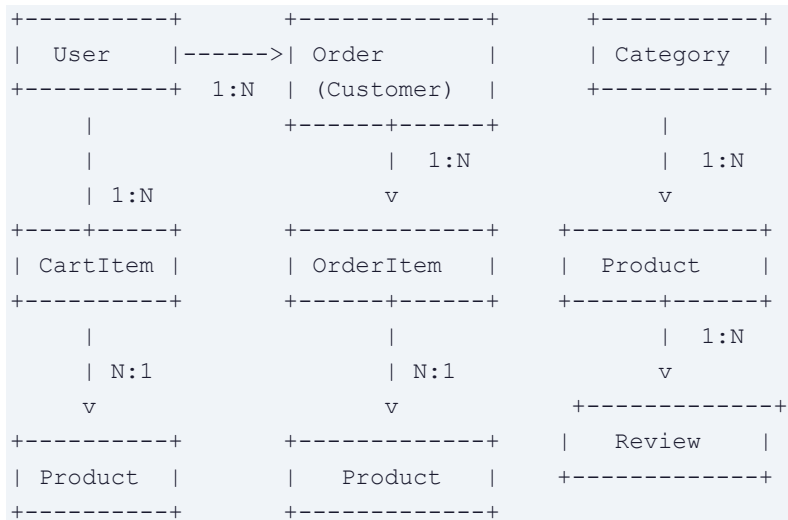
8. Database Design

8.1 Entity Summary

Table 9: Database Entity Summary

Table / Entity	Key Attributes	Purpose
User	Id, Name, Email, PasswordHash, Role	Platform users across all three roles
Product	Id, Name, Description, Price, Stock, ImageUrl, CategoryId	Catalog items available for purchase
Category	Id, Name	Product grouping taxonomy
CartItem	Id, UserId, ProductId, Quantity	Persistent cart items per customer
Order	Id, UserId, Status, ShippingAddress, OTP, DeliveryUserId, RejectedBy, RejectionReason, RejectedAtUtc	Primary order record with lifecycle state
OrderItem	Id, OrderId, ProductId, Quantity, UnitPrice	Line items within an order
Review	Id, UserId, ProductId, Rating, Comment, CreatedAt	Customer product reviews
RefreshToken	Id, UserId, Token, ExpiresAt, IsRevoked	JWT refresh token storage

8.2 Entity Relationships



Additional relationships:

- User (Delivery role) -> Order: One delivery agent can be assigned to many orders (1:N via DeliveryUserId).

- User -> RefreshToken: One user can hold multiple refresh tokens (1:N).
- User -> Review: One user can write multiple reviews (1:N).

Figure 3: Database Entity Relationship Diagram

8.3 Order Status Enumeration

Table 10: Order Status Enumeration

Status Value	Description	Transition Trigger
Pending	Order created, awaiting admin action	Customer checkout
Assigned	Delivery agent assigned by Admin	Admin assigns delivery user
OutForDelivery	Delivery agent has picked up the order	Delivery agent status update
Delivered	Order successfully handed over with OTP	OTP verification by delivery agent
Rejected	Order cancelled by Customer, Admin, or Delivery	Rejection endpoint call

9. API Design

All API routes are prefixed with `/api`. The API follows RESTful conventions with role-gated access enforced via JWT claims.

9.1 Authentication Endpoints — `/api/auth`

Table 11: Authentication Endpoints (`/api/auth`)

Method	Endpoint	Access	Description
POST	<code>/register</code>	Public	Register a new user account
POST	<code>/login</code>	Public	Authenticate and receive JWT + refresh token
POST	<code>/refresh</code>	Public	Exchange refresh token for new JWT
POST	<code>/dev-reset-password</code>	Dev Only	Development utility — to be removed in production

9.2 Product Endpoints — `/api/products`

Table 12: Product Endpoints (`/api/products`)

Method	Endpoint	Access	Description
GET	<code>/</code>	Public	List all products with optional filters
GET	<code>/id</code>	Public	Get product details by ID
POST	<code>/</code>	Admin	Create a new product
PUT	<code>/id</code>	Admin	Update product details
DELETE	<code>/id</code>	Admin	Remove a product
POST	<code>/upload-image</code>	Admin	Upload product image

9.3 Order Endpoints — `/api/orders`

Table 13: Order Endpoints (`/api/orders`)

Method	Endpoint	Access	Description
POST	<code>/checkout</code>	Customer	Convert cart to order with shipping address
GET	<code>/my</code>	Customer	Retrieve all orders for current customer
GET	<code>/my/id</code>	Customer	Get specific order detail for current customer
GET	<code>/</code>	Admin	List all platform orders
POST	<code>/id/assign-delivery</code>	Admin	Assign a delivery agent to an order
POST	<code>/id/reject</code>	Customer	Customer rejects an order with a reason

Method	Endpoint	Access	Description
POST	/id/reject-admin	Admin	Admin rejects an order with a reason
GET	/delivery-users	Admin	List all available delivery agents

9.4 Delivery Endpoints — /api/delivery

Table 14: Delivery Endpoints (/api/delivery)

Method	Endpoint	Access	Description
GET	/orders	Delivery	Get all orders assigned to current agent
POST	/orders/{id}/status/{status}	Delivery	Transition order status
POST	/orders/{id}/verify-otp	Delivery	Submit OTP to complete delivery
POST	/orders/{id}/reject	Delivery	Reject an assigned order with reason

9.5 AI Endpoints — /api/ai

Table 15: AI Endpoints (/api/ai)

Method	Endpoint	Access	Description
POST	/generate-description	Admin	Generate product description via AI
GET	/reviews-summary/{productId}	Admin	AI-generated summary of product reviews
POST	/smart-search	Public	Semantic product search via embeddings
GET	/recommendations/{productId}	Public	AI-powered product recommendations

10. Workflow & System Flow

10.1 Order Lifecycle Flow

```

Customer Checkout
|
v
[Order: Pending] <----- Stock deducted from product
|
| Admin assigns delivery agent
v
[Order: Assigned] <----- OTP generated, stored on order
|
| Delivery agent picks up
v
[Order: OutForDelivery]
|
| Delivery agent submits OTP
v
[Order: Delivered] <----- OTP verified, delivery complete

```

Figure 4: Order Lifecycle State Flow

10.2 Rejection Workflow

Any of the three roles may reject an order subject to the following rules:

- Orders with status Delivered cannot be rejected.
- Orders already in Rejected state cannot be rejected again.
- On rejection: OTP and delivery assignment are cleared, product stock is restored, and rejection metadata (actor, reason, timestamp) is persisted.

```

[Order: Pending / Assigned / OutForDelivery]
|
| Customer / Admin / Delivery calls reject endpoint
v
Validation checks pass
|
| RejectedBy, RejectionReason, RejectedAtUtc stored
| OTP cleared, DeliveryUserId cleared
| Product quantities restored to stock
v
[Order: Rejected]

```

Figure 5: Rejection Workflow State Flow

10.3 Delivery OTP Verification Flow

6. Customer places order. Order status is Pending.

7. Admin assigns a delivery user. OTP is auto-generated and stored.
8. Customer can view their OTP on the orders detail page.
9. Delivery agent marks order as OutForDelivery.
10. Delivery agent presents at delivery point, customer provides OTP verbally.
11. Delivery agent enters OTP in the delivery panel.
12. Backend validates OTP. On success, order status is set to Delivered.

10.4 Authentication Flow

13. User submits credentials to POST /api/auth/login.
14. Backend validates credentials, returns short-lived JWT access token and long-lived refresh token.
15. Frontend stores tokens in AuthContext.
16. Every API request includes the JWT as a Bearer token in the Authorization header.
17. When the JWT expires, the frontend automatically calls POST /api/auth/refresh with the refresh token.
18. Backend validates the refresh token, issues a new JWT, and rotates the refresh token.

11. Security Implementation

11.1 Authentication

- JWT Bearer token authentication is implemented using ASP.NET Core's built-in JWT middleware.
- Access tokens are short-lived (configurable via `Jwt__ExpiresInMinutes`).
- Refresh tokens are long-lived (configurable via `Jwt__RefreshTokenDays`), stored in the database, and can be revoked.
- Token issuance includes standard claims: user ID, email, and role.

11.2 Authorization

- All protected routes use `[Authorize]` and `[Authorize(Roles = "...")]` attributes.
- Role-based access is enforced at the controller and action level.
- Customer, Admin, and Delivery scopes are isolated; cross-role access is not permitted.

11.3 Password Security

- All passwords are hashed using BCrypt before storage.
- Plaintext passwords are never stored or logged.

11.4 Data Protection & Configuration

- Sensitive credentials (DB connection strings, JWT secrets, API keys) are stored in environment-specific `.env` and `appsettings.Local.json` files.
- These files are excluded from version control via `.gitignore`.
- Only safe template files (`.env.example`, `appsettings.Local.example.json`) are committed.

11.5 Runtime Protections

Table 16: Runtime Security Protections

Protection	Implementation
Rate Limiting	100 requests per minute per IP via ASP.NET Core built-in middleware
Global Exception Handling	Centralized middleware catches unhandled exceptions and returns structured error responses
API Response Wrapping	Consistent response envelope applied via response filter
CORS Policy	Restricted to frontend origins <code>localhost:5173</code> and <code>localhost:5174</code> during development

12. AI Features

SmartBasket integrates with HuggingFace's hosted model APIs to deliver four distinct AI capabilities across the platform.

12.1 AI Smart Search

Endpoint: POST /api/ai/smart-search

Access: Public

Technology: HuggingFace Embedding Model

Traditional keyword search fails to capture user intent. Smart Search converts the user's natural language query into a vector embedding using the configured HuggingFace embedding model (with a configured fallback model). The resulting embedding is compared against pre-computed product embeddings to return semantically relevant products, even when exact keywords are absent.

- Example: Searching 'something to keep my coffee hot' returns thermos and insulated mug products.
- A fallback embedding model is configured in case the primary model is unavailable.

12.2 AI Product Recommendations

Endpoint: GET /api/ai/recommendations/{productId}

Access: Public

Technology: HuggingFace Embedding Model

On a product detail page, the recommendations engine computes the embedding of the current product and finds the most semantically similar products in the catalog, returning a curated list of related items to encourage further discovery.

12.3 AI Description Generator

Endpoint: POST /api/ai/generate-description

Access: Admin

Technology: HuggingFace Text Generation Model

Admins can provide basic product attributes (name, category, key features) and receive an AI-generated product description. This reduces the manual effort required during product catalog expansion and ensures consistent description quality.

12.4 AI Review Summarizer

Endpoint: GET /api/ai/reviews-summary/{productId}

Access: Admin

Technology: HuggingFace Text Generation Model

For any product, the Admin panel can request an AI-generated summary of all customer reviews. The AI distills sentiment, common praise, and recurring complaints into a concise narrative — enabling administrators to quickly assess product reception without reading every individual review.

13. Implementation Details

13.1 Repository Pattern

Data access is abstracted behind repository interfaces, decoupling the service layer from the EF Core DbContext. This enables testability and allows the underlying data store to be swapped without changing business logic.

13.2 DTO Contract Design

The API surface is defined using Data Transfer Objects. Request DTOs validate incoming payloads. Response DTOs shape outgoing data, preventing over-exposure of entity internals (such as password hashes or raw database IDs).

13.3 Startup Seeding

On application startup, a seeder runs to ensure base data (such as default users, categories, or test products) is present in the database. A repair task also runs to correct any data integrity issues introduced during development iterations.

13.4 React Query for State Management

All server-side state — product lists, order data, cart contents — is managed via TanStack React Query. This provides automatic background refetching, cache invalidation, loading and error states, and stale data management without manual Redux boilerplate.

13.5 OTP Generation Strategy

OTPs are generated server-side at the time of delivery assignment and stored on the Order record. The OTP is exposed to the customer on their order detail view, and submitted by the delivery agent at handover. The backend validates the submitted OTP against the stored value before completing the delivery.

13.6 Stock Restoration on Rejection

When an order is rejected, the backend iterates over its OrderItems and increments the stock quantity of each associated Product accordingly. This ensures inventory accuracy without requiring a separate stock management job.

14. User Interface Overview

The frontend is a single-page application with three distinct panel experiences, all accessed through a unified layout with role-aware navigation.

14.1 Public Storefront

- Landing page with featured products and category navigation.
- Product listing page with category filter controls.
- Product detail page with full description, pricing, stock status, and customer reviews section.
- AI Smart Search page with a natural language input field and semantically matched results.
- Login and Registration pages with form validation.

14.2 Customer Panel

- Cart page displaying all items with quantity controls and order total.
- Checkout page with shipping address form.
- Orders list page showing all orders with status badges.
- Order detail page showing items, tracking status, OTP (when applicable), and rejection history.

14.3 Admin Panel

- Dashboard with platform KPIs and summary metrics.
- Categories management with creation form.
- Products management with full CRUD interface and image upload.
- Orders management with assignment controls, rejection actions, and status filtering.
- Past Deliveries historical table.
- AI Tools page with description generator and review summarizer interfaces.

14.4 Delivery Panel

- Assigned Orders queue with current order cards.
- Status update controls and OTP input form per order.
- Past Deliveries history view.

15. Testing

15.1 API Testing

- Swagger UI (available at /swagger) was used for endpoint-by-endpoint functional testing throughout development.
- All API routes were exercised with valid and invalid payloads to verify error handling, authorization guards, and business rule enforcement.
- Rejection workflow edge cases (double rejection, rejecting a delivered order) were explicitly tested.

15.2 Role-Based Access Testing

- Each role's JWT token was independently tested against endpoints requiring other roles.
- Confirmed that 403 Forbidden responses are returned for cross-role access attempts.

15.3 Frontend Functional Testing

- Manual end-to-end flows were executed: register -> login -> browse -> add to cart -> checkout -> track order -> delivery assignment -> OTP verification -> completion.
- Rejection flows tested from all three role perspectives.

15.4 Known Test Gaps

- No automated unit tests or integration tests are currently implemented.
- No automated end-to-end test suite (e.g., Playwright or Cypress) has been configured.
- These are identified as priority items for future development iterations.

16. Limitations

Table 17: Project Limitations Summary

#	Limitation	Impact
1	No automated test suite	Regression risk increases as features are added
2	CORS restricted to localhost	Requires reconfiguration for any production deployment
3	Frontend bundle chunk-size warning	May affect initial page load time in production; no functional impact
4	Nullable warnings in backend	Nullable is disabled in compilation; some null-safety gaps exist in model files
5	dev-reset-password endpoint	Development utility must be removed before any production deployment
6	No payment gateway	Platform does not process real payments; checkout is order-creation only
7	No email / notification system	No transactional emails for order confirmation, OTP delivery, or rejection alerts
8	AI latency dependency	Smart search and recommendations are dependent on HuggingFace API response times

17. Future Scope

The following enhancements are planned for subsequent iterations:

19. Automated Testing: Implement xUnit unit tests for service and repository layers; add Playwright or Cypress end-to-end test suite.
20. CI/CD Pipeline: Configure GitHub Actions or Azure DevOps pipelines for automated build, test, and deployment on push to main.
21. Payment Gateway Integration: Integrate Razorpay or Stripe to support real payment processing at checkout.
22. Email Notification System: Send transactional emails (order confirmation, OTP, rejection notice) via SendGrid or SMTP.
23. Production Hardening: Enable HTTPS, tighten CORS for production origins, remove development utilities, rotate all secrets.
24. API Versioning: Introduce /v1/ prefixed routing to support backward-compatible API evolution.
25. Product Search Caching: Cache embedding vectors for products to reduce AI API call volume and latency.
26. Admin Analytics Dashboard: Time-series charts for revenue, order volume, and product performance.
27. Wishlist Feature: Allow customers to save products for future purchase consideration.
28. Mobile Application: Develop a React Native mobile client consuming the same backend API.

18. Conclusion

SmartBasket demonstrates the successful design and implementation of a full-stack, role-based e-commerce platform with production-level architectural discipline. The project addresses real-world commerce challenges — multi-role access control, order lifecycle management, OTP delivery verification, rejection audit trails, and AI-enhanced product discovery — through deliberate engineering decisions applied consistently across the backend, frontend, and database layers.

The backend's layered architecture (Controllers > Services > Repositories > DTOs) provides clean separation of concerns and a foundation for testability and maintainability. The frontend's feature-first modular structure, combined with React Query for server state, delivers a responsive and predictable user experience across all three operational panels.

The AI integrations elevate SmartBasket beyond a conventional CRUD application, showcasing how modern language model APIs can be practically applied within a commerce context to improve search relevance, automate content generation, and distill user feedback.

With the identified future scope items — automated testing, CI/CD, payment integration, and production hardening — SmartBasket is well-positioned to evolve from a portfolio-grade demonstration into a deployable production system.

— End of Document —